

Executive Report: Cost Savings & Reliability Risk

Observation window: Sept 1-15, 2026 (15 days) | Sizing profile: production | Prepared: Sept 15, 2026

Executive Summary

A fleet-wide review of container CPU and memory requests across all connected clusters identified \$3,353/month (\$40,238 annualized) in rightsizing opportunity across 314 over-provisioned containers, out of 732 analyzed. The opportunity is concentrated: the top 10 workload types account for 79% of identified savings.

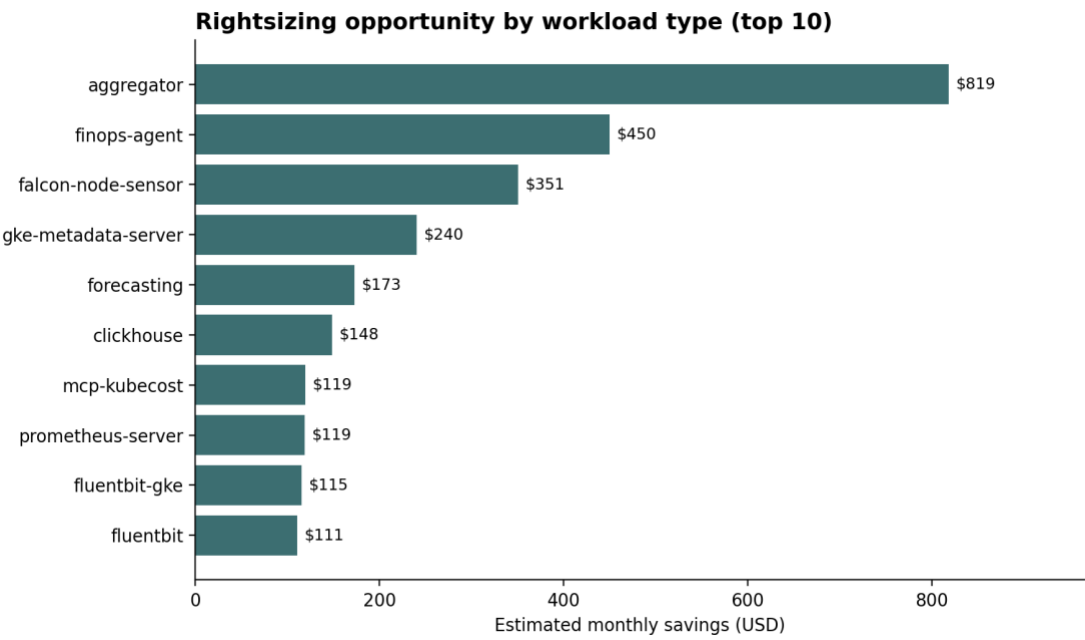
\$3,353 Monthly savings opportunity	\$40,239 Annualized	314 Containers over-provisioned
---	-------------------------------	---

268 Containers under-provisioned (risk)	\$351 / mo Largest single fix	732 Containers analyzed
---	---	-----------------------------------

- Quick win: the CrowdStrike Falcon sensor DaemonSet is running at 2% CPU efficiency and is a single-change, \$351/month fix.
- Systemic finding: the shared "finops-agent" default resource template (500m CPU / 512Mi memory) is miscalibrated in both directions across 35+ deployments — CPU is over-requested (4-20% utilization) while memory is simultaneously under-requested (100-165% utilization in many deployments). Fixing the shared Helm chart defaults addresses both cost and reliability risk in one change.
- Reliability: 268 containers are separately flagged as under-provisioned. These are not cost opportunities — they are OOM/throttling risk and should not be reduced further.
- Governance gap: kube-proxy pods on multiple clusters have no memory request configured at all (0Mi), leaving them without meaningful QoS protection under node memory pressure.

Where the Savings Are

Savings are dominated by a small number of workload types rather than being spread evenly across the fleet. Kubecost's own agent components (aggregator, finops-agent, forecasting, mcp-kubecost) account for over half of total identified savings — largely in non-production QA, demo, and test namespaces running production-sized defaults.



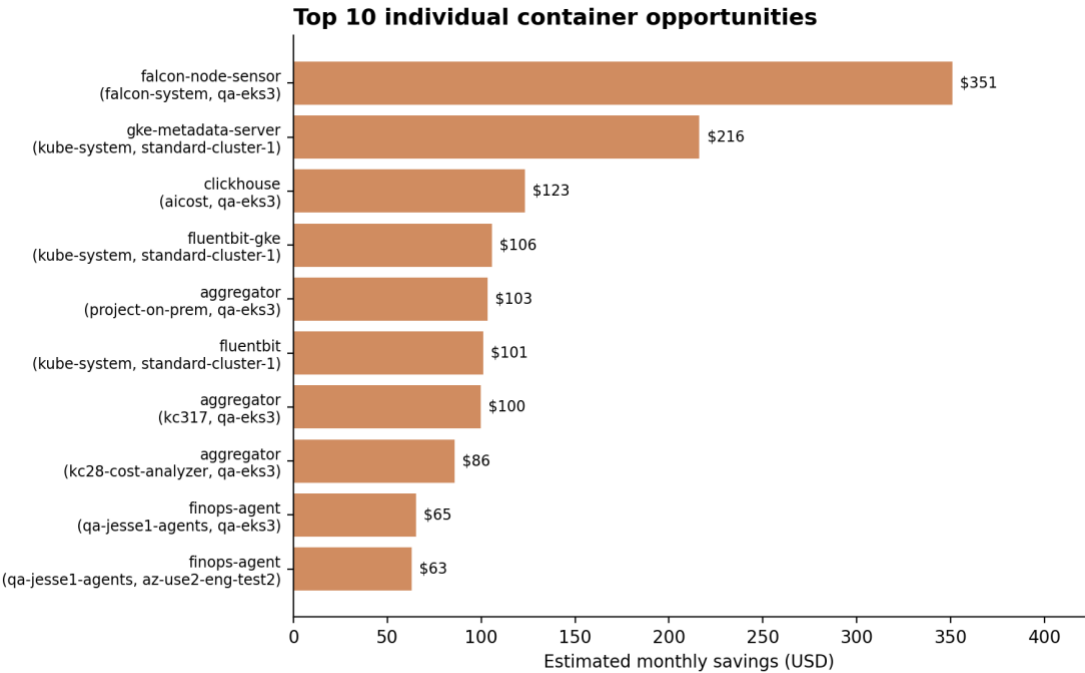
Top 15 Individual Opportunities

Cluster	Namespace	Container	Current req (CPU/Mem)	Recommended	Monthly \$ saved
eks-cluster1	falcon-system	falcon-node-sensor	801m / 5250Mi	28m / 2647Mi	\$350.92
standard-cluster-1	kube-system	gke-metadata-server	100m / 100Mi	10m / 57Mi	\$216.23
eks-cluster1	aicost	clickhouse	5000m / 16384Mi	340m / 6146Mi	\$123.35
standard-cluster-1	kube-system	fluentbit-gke	50m / 100Mi	10m / 49Mi	\$105.81
eks-cluster1	project-on-prem	aggregator	3468m / 10099Mi	93m / 7586Mi	\$103.49
standard-cluster-1	kube-system	fluentbit	50m / 100Mi	12m / 53Mi	\$101.05
eks-cluster1	kc317	aggregator	2000m / 20480Mi	10m / 716Mi	\$99.70
eks-cluster1	kc28	aggregator	3488m / 9952Mi	676m / 7930Mi	\$85.96
eks-cluster1	qa-test11-agents	finops-agent	500m / 512Mi	81m / 940Mi	\$65.19
az-use2-eng-test2	qa-test11-agents	finops-agent	2000m / 20480Mi	10m / 262Mi	\$63.11
eks-cluster1	kc-eng-bucket-agents2x-primary	aggregator	1000m / 15360Mi	105m / 2651Mi	\$60.64
dev-1	qa-test11-agents	finops-agent	2000m / 20480Mi	10m / 274Mi	\$54.57
standard-cluster-1	kubecost-turbo-test-2	aggregator	500m / 2048Mi	11m / 79Mi	\$48.68

Cluster	Namespace	Container	Current req (CPU/Mem)	Recommended	Monthly \$ saved
kc-demo-2025	monitoring	memcached	500m / 9830Mi	10m / 237Mi	\$47.82
eks-cluster1	kube-system	ebs-plugin	132m / 74Mi	10m / 54Mi	\$41.97

Full detail: 314 candidates across 90+ workload groups; contact FinOps for the complete export.

Top 10 Individual Containers, Visualized



Reliability & Governance Risk

268 containers show usage above their current request — these represent OOM-kill and throttling risk, not savings. The table below highlights the most material findings. None of these should be downsized; several need a request increase instead.

Cluster / namespace	Container	Signal	Current request	Observed usage (avg/max)
standard-cluster-1 / kc-eng-bucket-agents2x	cost-model	Memory usage 391% of request	201m / 55Mi	216 / 269 MiB
standard-cluster-1 / infra-kubecost	cost-model	Memory usage 176% of request	201m / 55Mi	97 / 158 MiB
standard-cluster-1 / emailer	emailer	Memory spikes to 516% of request	101m / 55Mi	6 / 253 MiB
standard-cluster-1 & eks-cluster1 / kube-system	kube-proxy	No memory request configured (0Mi) on multiple clusters	101m / 0Mi	35–40 / 49–61 MiB
35+ namespaces / "-finopsagent" deployments	finops-agent	Memory usage 100–165% of the 512Mi default request	500m / 512Mi	470–730 / 680–891 MiB

Cluster / namespace	Container	Signal	Current request	Observed usage (avg/max)
kc-demo-1 / kubecost-v3	aggregator	Memory usage 121% of request; CPU also under-sized	6000m / 12288Mi	6258 / 14940 MiB

Note: these figures compare usage to requests, not to configured limits, and cannot see CPU throttling, OOM history, or QoS class. Verify each workload's limit and QoS class before adjusting requests.

Recommended Actions

Action	Est. monthly impact	Effort	Risk / benefit addressed
Rightsize the CrowdStrike Falcon sensor DaemonSet CPU/memory requests (eks-cluster1)	\$351	Low	Single config change, cluster-wide effect
Fix the shared finops-agent Helm chart defaults: cut CPU request from 500m toward ~50-100m, raise memory request from 512Mi toward ~800Mi-1Gi	\$450+ / month, plus OOM risk reduction across 35+ deployments	Medium	Cost waste and under-provisioning fixed in one PR
Rightsize platform DaemonSets: gke-metadata-server, fluentbit/fluentbit-gke, aws-node, kube-dns	~\$400	Medium	Requires platform/SRE sign-off; affects every node
Decommission or rightsize stale test ClickHouse / aggregator pods in non-prod namespaces (test2n-ch-ha, kc317, kc28, test3-testbed, kcm-cloud-dev, etc.)	~\$700+	Low-Medium	Likely abandoned test infrastructure — confirm before deleting
Add explicit memory requests to kube-proxy across GKE/EKS clusters	Minimal direct \$	Low	Closes a QoS/governance gap; improves eviction behavior under node pressure
Set a namespace ResourceQuota policy for non-prod namespaces using the "development" sizing profile	Prevents recurrence	Medium	Stops future test/demo workloads from inheriting production-sized defaults
Validate the 6 flagged under-provisioned production-adjacent workloads (cost-model, emailer, kubecost-v3 aggregator) before any downsizing elsewhere touches them	N/A	Low	Prevents introducing OOM kills while pursuing savings

Technical Appendix

How the Sizing Numbers Are Calculated

For each container, Kubecost samples CPU and memory usage across the 15-day window, rolls the samples up to one value per day, then applies a quantile across those daily values. CPU and memory are deliberately handled differently because they fail differently under contention.

- CPU — algorithm `quantileOfAverages`: for each day, Kubecost computes the container's average CPU usage across that day, then takes the P80 quantile (`q_cpu = 0.80`) across those daily averages. Averaging within a day smooths transient spikes. This is acceptable because CPU is compressible — a container that exceeds its CPU request is throttled, not killed.

Recommended CPU request = $P80(\text{daily avg usage}) \div \text{target_cpu_utilization} (0.65)$

- Memory — algorithm `quantileOfMaxes`: for each day, Kubecost takes that day's maximum memory usage, then takes the P95 quantile (`q_ram = 0.95`) across those daily maxima. Memory is incompressible — exceeding a limit gets a container OOM-killed outright — so sizing anchors to peaks, not averages, at a higher quantile than CPU.

Recommended memory request = $P95(\text{daily max usage}) \div \text{target_ram_utilization} (0.65)$

- In both formulas the divisor is intentional: dividing the quantile by 0.65 inflates the recommended request so the workload runs at roughly 65% utilization of its new request, not pinned at the edge of it.
- Dollar savings: $\text{monthlySavings} = (\text{current_request} - \text{recommended_request}) \times \text{the cluster's effective hourly unit price for that resource (from Kubecost's cost model — cloud billing-based or custom pricing)} \times \text{hours in a 30-day month, summed across CPU and memory.}$ This is a request-level figure, not an invoice-level one — see caveats below.

OOM-Kill Risk: the Mechanics Behind the Risk Table

The `currentEfficiency` and `pct_change_memory` figures describe exposure to two distinct kill paths in Kubernetes, and the risk table's findings map onto them differently:

- Hard OOM kill (cgroup limit): if a container's memory usage exceeds its configured limit — not its request — the kernel cgroup OOM killer terminates it immediately, regardless of node-wide pressure. This dataset only compares usage to requests; Kubecost's rightsizing API can't see configured limits. A row showing 165% of request could be safely under a generous limit, or could already be getting killed if $\text{limit} \approx \text{request}$. This is why every risk-table row should be checked against its actual limit before acting.
- Node-pressure eviction (kubelet, request-relative): under node-wide memory pressure, the kubelet ranks pods for eviction using an `oom_score_adj` that is set by QoS class and, within Burstable, scales with how far usage exceeds request:
- *Guaranteed (request = limit for both CPU and memory): `oom_score_adj` fixed near -997 — evicted only as an absolute last resort.*
- *Burstable (a mix of requests/limits, or request < limit): `oom_score_adj` climbs roughly with $(\text{memory usage} \div \text{memory request})$ — exactly the ratio `currentEfficiency_memory` reports. The higher that ratio, the more evictable the pod becomes.*
- *BestEffort (no request or limit set on any resource): fixed at 1000, the maximum — evicted first, always, regardless of actual usage.*

Two risk-table entries sit at opposite ends of this mechanism for the same root cause — a resource-governance gap — and are worth distinguishing:

- cost-model containers (currentEfficiency_memory 1.76–3.91): Burstable pods running at 176–391% of their memory request. Not necessarily near a hard OOM kill — that depends on the configured limit — but their oom_score_adj rises with every MiB above the request, making them progressively more likely to be picked first when any pod on the node needs memory back.
- kube-proxy (memory request = 0Mi): a zero request doesn't mean "unmonitored" — it means the usage-to-request ratio the kubelet uses for eviction scoring is undefined, and in practice treated as maximally unfavorable. kube-proxy becomes one of the most evictable processes on the node by construction, independent of how little memory it actually uses (35–61 MiB observed here). Setting even the conservative request this data recommends (~90MiB) fixes the scoring, not just the visibility.

Net effect: the remediation is identical for both patterns — set a memory request that reflects real peak usage — but the failure mode differs (creeping eviction priority vs. undefined eviction priority), which is why both are flagged despite looking very different in the raw efficiency numbers.

Methodology & Caveats

- Source: Kubecost container request-sizing recommendations, 15-day observation window (Sept 1-15, 2026), "production" sizing profile (CPU sized to P80/quantileOfAverages, memory to P95/quantileOfMaxes, targeting 65% utilization).
- Figures represent request-level opportunity, not a guaranteed invoice reduction. Freed capacity becomes real savings once pods repack and nodes are scaled in — pair rightsizing with the node-group sizing recommendations to realize it.
- Replicas and revisions of the same controller are pooled into one distribution per container name; per-cluster detail is available on request.
- Before applying any CPU reduction, confirm the workload is not already being throttled. Before applying any memory reduction, confirm the pod is not "Guaranteed" QoS (request = limit), since changing the request alone changes that class.